Computer Communications 225 (2024) 171–184

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

Training a Graph Neural Network to solve URLLC and eMBB coexisting in 5G networks

Seyyed Mohammad Mahdi Hosseini Daneshvar, Sayyed Majid Mazinani *

Department of Electrical and Computer Engineering, Imam Reza International University, Mashhad, Iran

ARTICLE INFO

Keywords:
5G NR
URLLC
eMBB
Puncturing
Graph Neural Network
Resource allocation

ABSTRACT

Coexistence of enhanced mobile broadband and ultra-reliable low latency communication in 5G networks is a challenging problem due to the conflicting requirements. In this paper, we decompose the problem into eMBB and URLLC resource allocation phases. For the first phase we propose a heuristic algorithm with $\mathcal{O}(n)$ runtime and prove its efficiency and optimality under min–max fairness paradigm. For the URLLC resource allocation, the puncturing framework is adopted and a novel approach using the Graph Neural Networks is proposed to maximize eMBB data rates and fairness while minimizing URLLC outage probability. We show that the runtime of this GNN-based algorithm is also $\mathcal{O}(n)$. To train the GNN, an application-specific loss function is designed and empirically shown to be convergent. Our simulation results show that our proposed approach performs very well in terms of eMBB data rates, fairness, and URLLC outage probability in comparison to a number of thoughtfully chosen baselines. We also demonstrate that the proposed GNN is robust to changes in network topology and traffic volume. As we show our algorithm has $\mathcal{O}(n)$ runtime, it is fully practical for solving the resource allocation problem in the very short time spans that are required by 5G and future generation networks.

1. Introduction

The 5th generation New Radio (NR) offers three services to support state-of-the-art applications in wireless networks like autonomous cars, remote surgery, and virtual reality. These services include ultra-reliable low-latency communication (URLLC), massive machine-type communication (mMTC), and enhanced mobile broadband (eMBB). The URLLC service prioritizes extreme reliability and low latency for mission-critical applications such as remote surgery and autonomous cars, while mMTC requires high connection density and energy efficiency for IoT devices sending data sporadically with a low and fixed uplink transmission rate [1–3]. eMBB aims to deliver high data rates and an overall enhanced user experience, with high-resolution video streaming and virtual reality as examples of eMBB traffic.

To meet these diverse requirements, the 3GPP standard body proposes a co-existing mechanism for URLLC and eMBB based on a superposition or puncturing framework. Similar to LTE, time is divided into Transmission Time Intervals (TTIs) of 1 ms, with each TTI subdivided into mini-slots (sTTI) to support low-latency URLLC communication. The duration of a mini-slot may vary from 0.125–0.5 ms depending on the OFDMA numerology used. For more details on OFDMA numerology in 5G NR standards see [4]. At the beginning of each TTI, the next-generation node B (gNB) assigns time–frequency rectangles known as Resource Blocks (RB) to eMBB users. However, due to time-varying nature of channels in wireless networks, the peak data rate for each user may differ on each RB over time [5]. The standard does not specify a particular scheduling method but aims to maximize channel utilization for users by allocating advantageous RBs to each user equipment (UE) on time and frequency domains. To achieve this, the gNB might use Channel State Information (CSI) to collect information and make resource allocation decisions that opportunistically assign RBs to UEs [6].

In order to meet the stringent latency requirements of 0.25–0.3 ms per packet for short and sporadic URLLC packets, it is necessary to schedule them on the immediate mini-slot upon arrival. This requires puncturing RBs of some eMBB users, which allows for the URLLC traffic to cut through ongoing eMBB transmissions. However, in the superposition mode, assigning non-zero power to both transmissions can result in degraded rates for both URLLC and eMBB users [1,7].

To address this issue and achieve a reliability constraint of 99.999% for URLLC [8], a puncturing framework has been proposed. This framework involves cutting through eMBB transmissions by setting their power to zero during the short period of URLLC transmissions, resulting in a rate loss for the punctured eMBB users. At the end of TTI, the

* Corresponding author.
E-mail address: smajidmazinani@imamreza.ac.ir (S.M. Mazinani).

https://doi.org/10.1016/j.comcom.2024.07.008
Received 14 December 2023; Received in revised form 14 July 2024; Accepted 15 July 2024
Available online 18 July 2024
0140-3664/© 2024 Elsevier B.V. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

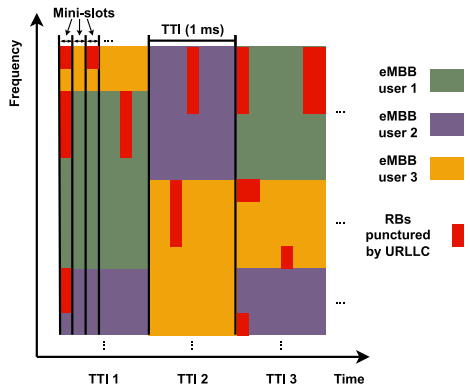


Fig. 1. A demonstration of URLLC puncturing.

gNB can inform these users of any puncturing that occurred during the timeslot [5].

The amount of URLLC traffic overlapped on RBs allocated to users throughout the timeslot determines the rate loss experienced by eMBB users when decoding this information. Fig. 1 illustrates this puncturing framework in the time–frequency plane [8]. By implementing this framework, it is possible to meet latency requirements while maintaining reliable communication for both URLLC and eMBB users [8].

The main challenge is to allocate RBs to eMBB users at TTI boundary while also scheduling on-demand URLLC traffic at the mini-slots boundary. This involves considering channel variations and puncturing RBs already assigned to eMBB users, all while maintaining reliability, throughput, and fairness [7]. The goal is to fulfill as many URLLC requests as possible with minimal negative effect on eMBB users. An alternative approach is orthogonal scheduling, where some RBs are reserved for URLLC in a semi-static or dynamic manner. However, this approach may result in wasted resources if there is no URLLC transmission [1]. In this study, we focus on the puncturing approach to address the co-existing problem of eMBB/URLLC and ensure URLLC constraints with minimal impact on eMBB users. Performance evaluation metrics include fairness data rate, and packet drop rate. A list of acronyms used throughout the text is given in Table 1.

1.1. Graph neural networks

Machine learning methods based on neural networks, have become a very successful tool to tackle various problem across multiple disciplines. Specially fully connected deep neural networks (DNN) are proven to be universal optimizers. However they suffer from lack of generalization and do not scale well when the parameters dimension grows [9–11]. Moreover, conventional deep learning architectures such as Convolutional and Recurrent Neural Networks (CNN and RNN) used in image and text processing applications will lose their power when applied to non-Euclidean data [12]. While many real world problems such as the ones in social and computer networks deal with non-Euclidean data that are best represented with a graph structure [13]. In addition to that, when using a DNNs in computer networks, the dimensions of input and output vectors are predetermined by the size of network and are fixed after the training, Deep Reinforcement Learning (DRL) methods also suffer form this issue [14]. It has been demonstrated that DNN methods generalized poorly when faced with networks of different size than that of the training phase [12]. One recent attempt to exploit graph topological data into deep neural network are the Graph Neural Networks. GNNs are an extension to the current neural network architectures that generalize CNNs [11,15].

In a GNN the traditional convolution filter is replaced with a so called Graph Convolution Filter. Message Passing Neural Network (MPNN) is a framework proposed to generalizes GNNs [16]. MPNN updates hidden states of the nodes by stacking multiple layers of graph convolution filters. Specifically, each node in the graph updates its hidden state based on the aggregated hidden states of its neighboring nodes [12]. The goal of a GNN is to learn the representation of nodes, edges or graphs with common task including node and edge regression and classification and graph classification and property prediction. [13]. GNNs offer the following advantages:

1. **Generalization and scalability**: In contrast to CNNs and DRL methods, a GNN trained on a graph with a number of nodes can be used effectively to predict the properties of a larger or smaller graph [9]. This is essential in solving computer network problems, as the size of the network graph is dynamic and re-training the model for every graph size is not practical.
2. **Permutation invariance**: The order of aggregating hidden state of nodes in a GNN does not affect the output features [15,17]. It is also shown in [18] that any graph optimization problem is permutation invariant.
3. **Exploiting Graph topology**: While fully connected neural networks and CNNs are suited for simply connected grids like pixels of an image, many real world problem include data point with complex relationship between them. It is best to incorporate these relationship in the architecture of the network [10,16].
4. **computational efficient**: Thanks to parallel execution, GNNS are efficient, It is very essential for next generation computer networks such as 5G as the delay requirements of these networks mandates finding an optimal solution in a very short time span. as an example GNNs are the only method than can solve the beamforming problem for thousands of users in milliseconds [12]

1.2. Motivation

Joint Resource allocation for eMBB and URLLC users is a very important problem in 5G networks. Existing state-of-the-art protocols have used DNN and DRL method that have succeeded in finding near-optimum solutions, but as mentioned before, these methods do not generalize and scale, are computationally heavy and do not exploit information topological structure of the network in their NN architecture, while GNN remedy these problems. It seems very promising to employ GNN methods in this context. GNN are have successfully been used in various problems in wireless networks and show very favorable results. Therefore in this paper, we aim to employ GNNs to solve the joint resource allocation of eMBB and URLLC users in 5G networks.

In our formulation, URLLC traffic indeed has higher priority over eMBB traffic. However, the challenge lies in the optimal assignment of incoming URLLC users to RBs that are already allocated to eMBB users. This assignment is non-trivial due to the following reasons. First, different RBs provide different data rates for each eMBB and URLLC user. Assigning an incoming URLLC user to any RB affects the data rate of the eMBB user currently using that RB and the data rate achieved by the URLLC user. Second, the problem is combinatorial in nature as it involves finding the optimal configuration among various possible permutations of assigning URLLC users to RBs, considering the impact on both eMBB and URLLC users.

GNNs can capture the complex relationships and dependencies between nodes, which are essential for understanding the impact of resource allocation decisions. They also efficiently handle large and dynamic graphs, making them suitable for real-time resource allocation in large networks. Using a GNN allows us to efficiently explore and evaluate different permutations of URLLC-to-RB assignments, ensuring that the overall network performance is optimized while prioritizing URLLC traffic. The GNN leverages its learned knowledge to predict the most effective allocation strategy, balancing the trade-off between eMBB and URLLC requirements.

1.3. Challenges

The presence of both URLLC and eMBB services using the same radio resource creates a difficult scheduling issue. This problem is not easy to solve because there is a trade-off between latency, reliability, and spectral efficiency that needs to be considered [1]. To achieve optimal resource allocation for 5G, it is necessary to use algorithms that are both fast and scalable. Speed is important due to the shorter time slots in 5G networks, while scalability is crucial because the number of user equipment (UE) is large and constantly changing. The simple method of statically allocating a portion of the bandwidth to URLLC users will lead to under-utilization of radio resources [5]. Traditional algorithm are time consuming and of high computational complexity. Hence, these global optimization algorithms are usually inappropriate for solving practical problems in wireless networks [19]. In practice, URLLC traffic is random and sporadic [1] which make dynamic and intelligent resource allocation a necessity. Although the machine learning based schemes can improve the time complexity and performance in wireless communication designs, the integration of wireless network topologies into the algorithms is still a challenge. Another challenge for machine learning based approaches is that The acquisition of a large training data set in real-world wireless networks is expensive or even impractical [19].

1.4. Contributions

The coexistence eMBB and URLLC in 5G networks is a challenging problem due to the conflicting requirements of high data rates and low latency. In this paper, we propose a novel approach using GNNs to maximize eMBB data rates and fairness while minimizing URLLC outage probability. To best of our knowledge this work is the first to incorporate GNNs for this problem. Our contribution can be summarized as:

- Proposing a heuristic algorithm with $\mathcal{O}(n)$ runtime for eMBB resource allocation in 5G networks, we prove its optimality according to the min–max fairness paradigm. We also prove a special case of the algorithm can find a feasible solution with upper-bound for eMBB data rate.
- for the URLLC puncturing problem, we propose a deep GNN architecture based on MLPs that solve the problem in $\mathcal{O}(n)$ and show its efficiency empirically. The proposed GNN models the network as a complete graph in which every RB is a node.
- As it is very hard or impractical to gather large labeled data sets from real wireless networks to train NNs [15], We propose an application specific loss function that enables us to train the network using simulated non-labeled data. The convergence of this loss function is shown empirically.
- We show that the GNN-based algorithm has runtime of $\mathcal{O}(n)$ which makes it a perfect solution for the ever shorter sTTI time spans of 5G and future generation mobile networks.
- We validate the performance and generalization of the our proposed approach. As opposed to traditional neural network, a GNN trained on a wireless network with n nodes can be effectively used on a smaller or larger network.

1.5. Organization

The rest of this paper is organized as follows. Related works are reviewed in Section 2. Sections 3 and 4 are dedicated to system model and problem definition respectively. The proposed approach is presented and discussed in the fifth section. Simulation and evaluations are enlisted in Section 6 and the conclusion is presented at the end.

Table 1
List of acronyms.

Acronym	Meaning
URLLC	Ultra Reliable Low Latency Communication
eMBB	Enhanced Mobile Broad Band
mMTC	Massive Machine Type Communication
3GPP	3rd Generation Partnership Project
LTE	Long Term Evolution
TTI	Transmission Time Interval
sTTI	Short Transmission Time Interval
RB	Resource Block
UE	User Equipment
CSI	Channel State Information
gNB	Next Generation Node B
GNN	Graph Neural Network
LP	Linear Programming
DLR	Deep Reinforcement Learning
MLP	Multi-Layer Perceptron
i.i.d.	Independent and Identically Distributed
CCDF	Complementary Cumulative Distribution Function

2. Related works

2.1. URLLC and eMBB coexisting in 5G

The standard for co-existing between URLLC and eMBB does not provide specific details on preemptive puncturing scheduling, leaving it up to the implementation. This presents a research opportunity, as recent studies have focused on optimizing network performance under this framework. One such study [1] formulated the resource allocation problem for eMBB and URLLC traffic as an optimization problem, with the goal of improving fairness, outage probability, reliability, and data rate. The eMBB resource allocation was broken down into three phases: RB allocation to users, power allocation, and URLLC scheduling with optimal punctured mini-slots per RB. While RB and power allocation were converted to a convex optimization problem, URLLC scheduling remained combinatorial with no closed form solution. To address this issue, a basic LP relaxation was used along with a DRL algorithm for scheduling in each mini-slot. The DLR algorithm utilizes a policy gradient actor critic model to select the number of punctured mini-slots for each RB. The reward function is determined by the URLLC users' outage probability. However, this method suffers from slow convergence due to DLR limitation.

Anand and colleagues [8] developed a framework that addresses the joint scheduling of URLLC and eMBB traffic. The framework utilizes three rate loss models, namely linear, convex, and threshold, to formulate the joint scheduling problem. For the linear model, a decomposition approach is proposed where the eMBB scheduler uses a greedy iterative gradient algorithm based on expected URLLC load while the URLLC traffic is randomly assigned to RBs at mini-slots. This approach is based on a theorem stating that any policy's throughput can be achieved by another policy with different eMBB to RB assignment and random URLLC placement. However, this approach does not apply to multi-URLLC user systems where each user's rate differs on each RB, requiring opportunistic resource allocation to push the system to its limit. In the convex model, an eMBB user's rate loss is a convex function of punctured mini-slots' number. mini-slot-homogenous policies are introduced for this model in which the URLLC placement policy remains fixed during a TTI. Overall, the framework proposed by Anand et al. provides a useful approach for joint scheduling of URLLC and eMBB traffic. However, it has some limitations such as the assumption of only one URLLC user and the focus on utility maximization for eMBB traffic. Future research could explore extensions of this framework to multi-user URLLC systems and consider other performance metrics such as delay and reliability. Additionally, incorporating machine learning techniques could potentially improve the efficiency and effectiveness of joint scheduling algorithms.

Table 2
Comparison of our approach with similar work.

Ref.	Features, Contributions, Advantages	Weaknesses	Method
[1]	Considers power allocation,	Slow convergence.	Decomposition and Relaxation method for Convex Optimization of the eMBB resource allocation and Policy Gradient Actor Critic Learning for URLLC allocation.
[5]	Heuristic algorithm for eMBB resource allocation, one-sided matching game for URLLC scheduling	Assigning one RB to each URLLC user.	Heuristic, Matching game
[6]	Model-free DRL based approach, continuous action state, approximates expected URLLC load, uses an information theory based algorithm to guarantee feasible solution based on the approximation.	Does not consider multiple URLLC users or opportunistic assignment of RBs based on their time-varying channel condition.	DRL
[20]	eMBB resource allocation problem is redefined based on min–max fairness and solved by a heuristic algorithm	Assumes single RB for each URLLC user, Does not consider channel gain of different URLLC users.	Heuristic
[21]	LP formulation for opportunistic eMBB resource allocation by employing CSI, geometric programming formulation for URLLC scheduling stage, URLLC requests can be queued for up to two mini-slots.	Does not take CSI for URLLC users.	LP, Geometric Programming
[22]	Exploiting data similarity for simultaneous transmission of URLLC and eMBB traffic, removing the need for informing eMBB users of puncturing, Deriving a closed-form expression for eMBB SER as a function of SNR.	No comparison with similar works or accepted baselines presented.	Heuristic
[23]	Puncturing based on one-to-one matching game to maximize eMBB MEAR while satisfying URLLC requirements, low complexity algorithm for eMBB resource allocation	Only compared simulation results to contract-based scheduling.	One-to-one matching game

The article [5] proposes an optimization problem aimed at maximizing the rate of eMBB users. To achieve this, the problem is divided into two sub-problems: eMBB and URLLC resource allocation. The eMBB resource allocation is addressed using a heuristic algorithm that takes into account the impact of URLLC on eMBB users up to timeslot $t-1$ when allocating resources in timeslot t. For URLLC allocation, a one-sided matching game is used where each URLLC user is matched with one eMBB user, while an eMBB user can be matched with multiple URLLC users. The preference of each URLLC user over eMBB users depends on the expected actual achievable rate until the last mini-slot. The system model considers multiple eMBB and URLLC users but assumes that a URLLC request requires only one RB for its operation, which may not be accurate due to channel variations that may require multiple RBs in a single mini-slot.

in [20], a comparable issue of maximizing the expected rate of eMBB over time is presented and divided into two sub-problems: eMBB and URLLC scheduling. The first sub-problem is redefined based on the min–max fairness principle, which aims to maximize the minimum expected rate of users over time. This sub-problem is solved using penalty successive upper-bound minimization (PUSM) or an alternative heuristic algorithm. To solve the second sub-problem, the transportation model with an algorithm based on minimum cell cost (MCC) is utilized. However, this study assumes a single RB per URLLC user and does not consider the channel gain of URLLC users.

Huang and colleagues [6] introduced a novel approach called DEMUX, which is a dual-stage, model-free DLR-based method aimed at minimizing eMBB utility loss while fulfilling all URLLC requests. The unique aspect of their approach is the integration of a continuous and large action space, which conventional learning algorithms like Q-learning cannot handle effectively due to their limited applicability to discrete and small action spaces. In the proposed architecture, a learning plane utilizes an improved deep deterministic policy gradient method to learn an optimal algorithm for preemption, while the scheduling plane uses this algorithm to compute the preemption solution for URLLC. However, since URLLC traffic is not known beforehand, an approximation for expected URLLC traffic is used, resulting in a non-guaranteed feasible solution. To address this issue, an information theory based formulation is employed that minimizes the relative entropy between the desired feasible solution and the current unfeasible solution. This problem is converted into a convex optimization problem that can be solved using standard convex optimizers or a faster heuristic algorithm suggested by the authors. It should be noted that DEMUX's system model does not consider multiple URLLC users or opportunistic assignment of RBs based on their time-varying channel condition.

In [21], an optimization formulation is presented with the objective of maximizing the total throughput of eMBB users, while ensuring that the URLLC drop rate remains below a predetermined threshold. This problem is divided into two stages: eMBB scheduling and URLLC placement. The first stage involves solving a linear integer programming problem, while the second stage is converted into a geometric programming form. Additionally, to minimize packet drops, URLLC packets can be queued for up to two mini-slots. The protocol assumes that CSI is known and assigns eMBB users to the best RBs opportunistically. However, it does not take into account the channel state information for URLLC users. A more effective protocol would differentiate between URLLC users and utilize CSI to meet their throughput requirements with minimal impact on eMBB users.

In [22] a method of puncturing for simultaneous transmission of URLLC and eMBB traffic by exploiting data similarity is proposed. The objective of the scheduler is to reduce the size of punctured eMBB symbol sequences by puncturing the eMBB RBs that have highest similarity with the on-demand URLLC request. As the goal the puncturing strategy is to minimize the impact of simultaneous transmission on eMBB and URLLC, it is proposed that the eMBB users do not need to be informed about the puncturing. This enables the BS to avoid the

costly puncturing indicator signal. Therefore the retransmission-based puncturing is replace with code-based puncturing. They also derive a closed-form expression for the symbol error rate (SER) of the eMBB traffic concluding that the eMBB SER is a function of signal-to-noise ratio (SNR), the average URLLC load, and the average similarity. finally the feasibility of the proposed approach is demonstrate through several numerical examples.

Prathyusha and Sheu [23] Proposed a puncturing and superposition based approach via one-to-one matching game model. Their goal is to maximize the minimum expected achieved rate (MEAR) while also satisfying the URLLC service requirements. They provide a formulation for the resource optimization problem and decompose in into eMBB and URLLC resource allocation phases. A low complexity algorithm is proposed for the eMBB resource allocation to improve throughput and MEAR. For URLLC scheduling either super-position or puncturing technique is adopted based on the network condition with the goal of retaining fairness among eMBB users. They also present the simulation results which indicate the optimality of their approach contract based scheduling [24]. A brief comparison of related works is presented in Table 2.

2.2. Comparison of related works

In this section a comparison between our work and the reviewed similar works is presented with regard to different aspects.

2.2.1. System model

The system model of this works is similar to from [1]. One main point of difference the works is the assumption of number of RBs per URLLC users, while some works assume only one RB for each URLLC user [5,6,20], this work and others [1,22,23] allow a URRLC UE to use multiple RBs to satisfy its requirements.

2.2.2. Traffic model

The common traffic model for the eMBB users in similar works is the fully buffered model [1,5,6,20,21,23]. On the other hand, URLLC requests are assumed to follow a poison distributed inter-arrival time [1,5,6,20,23]. This work uses these two commonly used models to generate traffic for conducting the simulations. Some works [21] also consider the queuing of the URLLC user for a few mini-slots. In this work we do not consider queuing of URLLC packets.

2.2.3. Evaluation

Simulation is the main tool for evaluating the performance in the literature. The common simulation tools include Matlab [6,22] and Python, which we have utilized in this study.

2.3. GNN in wireless networks

The authors of [25] provide a comprehensive overview of the interplay between GNNs and wireless communication. They investigate two aspects of this interactions, namely, GNN for wireless communication and (GNN4COM) and wireless communication for GNN (COM4GNN). The first aspect is about applying GNNs to solve problems in wireless networks while the second aspect concerns with developing wireless mechanisms for practical implementation of decentralized GNNs in the presence of imperfections and random noise in communication. While their work provides a broad overview, our research narrows down the focus to resource allocation problems and presents specific solutions and performance evaluations for eMBB and URLLC environments. Our contribution lies in the detailed problem modeling and the use of GNNs to solve the eMBB/URLLC resource allocation problem efficiently, which is a more targeted application compared to the general review provided by [25]. A notable contribution applying GNNs to problems in wireless networks is the meta-gating framework [26]. This framework addresses the challenges of changing CSI distributions in real-world wireless communication by combining model-agnostic meta-learning (MAML) with an unsupervised learning mechanism. The framework comprises of a dual network architecture in which one network adapts with varying CSI while the other network dynamically adjust the parameters of the first network. This approach aims to achieve seamless, quick, and continuous adaptation to the dynamic wireless environment.

Chen et al. [15] have proposed a supervised learning method for training a GNN to allocate resources in 5G Device-to-Device (D2D) IoT networks. The model presented in this paper represents desirable communication links as nodes and harmful inference links as edges. To solve resource allocation problems involving integer parameters in wireless communications, the authors demonstrate the effectiveness of a multi-layer GNN network with DNN aggregation and combination operators. The proposed framework is tested on link scheduling and joint power and channel allocation problems in D2D networks. The runtime efficiency of the GNN model makes it suitable for real-time wireless IoT systems.

In [11], a centralized approach was presented that combines a GNN with a reinforcement learning framework to address resource allocation in wireless control systems for industrial applications. By taking fading and interference into account, it is possible to model the wireless network as a graph. Then, by using a standard RL framework, the weights of the graph convolution filters can be learned. With this approach, it is possible to mitigate the disadvantages of traditional RL-based methods, such as high sample complexities, a lack of generalization and scalability, and a large learning space dimensionality.

An adaptive power allocation scheme for Device-to-Device (D2D) wireless networks that seeks to maximize Quality of Service (QoS) have been proposed in [9]. The corresponding QoS objective function has been formulated using Lagrangian dual domain and then optimized through the utilization of three separate GNNs to achieve scalability over various network sizes.

Rusek et al. [10,16] demonstrated a successful utilization of GNNs to model Software Defined Networks (SDNs), with the aim of optimizing routing. The developed model is able to predict latency and jitter between any source and destination, which can be used to optimize and plan routes. The capability of the trained model to generalize to previously unseen network sizes and topologies was also verified through empirical results.

In [12], a comprehensive study of GNNs and their practical capabilities in wireless networks was presented. The authors demonstrated that GNNs can achieve near-optimal performance with fewer training samples; specifically, they showed that the number of training samples needed can be reduced to $\mathcal{O}(n^2)$ while providing an $\mathcal{O}(n)$ lower generalization error compared to Multi-Layer Perceptrons. Furthermore, they devised a general framework for GNNs to be used to address a wide range of design problems in wireless networks, which was then successfully employed to deal with three specific issues: power control in both Device-to-Device and cell-free networks, and the hybrid precoding in millimeter Wave Communication. This work provides an insightful understanding of employing GNNs for wireless communication challenges. Our contribution extends theoretical framework of [12] by applying it to specific use-cases in eMBB and URLLC, demonstrating practical implementation and performance improvements in these scenarios.

In [27], GNNs are used in the context of Digital Twin technology to achieve efficient network slicing in 5G industrial IoT networks. The authors introduce a new model for accurately predicting the end-to-end latency of network slices based on a graph representation of the underlying physical infrastructure. This model is capable of generalizing and making accurate predictions under various conditions, including changes to topology, slice deployment, and traffic distributions.

In [14], a routing protocol for Wireless Sensor Networks (WSNs) based on GNNs is proposed. A deep reinforcement learning agent is trained, using a GNN instead of CNNs. This model is able to identify

the most efficient routing strategy even in previously unseen network topologies.

In [28], a cross-layer optimization scheme for Multi-Path Transmission Control Protocol (MP-TCP) routing for 5G networks utilizing GNNs is presented. A GNN model is employed to predict the throughput of arbitrary topologies, in order to aid in multipath routing decisions. Furthermore, an additional GNN model combined with SDN is proposed to optimize routing.

Liu et al. [29] address the issue of Age-of-Information (AoI) in interference-limited networks by utilizing a combination of DLR and graph neural network. They propose an actor and critic deterministic policy gradient approach to determine the optimal policy, which involves two GNNs for the actor and critic respectively. The actor is a random edge GNN that maps problem's input to the solution, while the critic is a message-passing GNN that estimates the long-term reward of state–action pairs visited by the policy. The received interference is considered as an aggregated message of a node in the message-passing GNN. Simulation results demonstrate that this method can effectively handle a variable number of wireless links in the communication network.

In conclusion, while previous works have laid the groundwork for applying GNNs to wireless communications, our research advances the field by providing a new, practical solution for resource allocation in eMBB and URLLC scenarios. Our method leverages the strengths of GNNs to handle the complexities of dynamic wireless environments, offering improved performance and adaptability. For an overview on the applications of GNNs in wireless networks refer to [17].

3. System model

The system model we employ is the same as the one presented in [1] and similar to other works [5,8,20,21]. We consider downlink requests in a wireless network consisting of a base station (gNB), a set $\mathcal{K}$ of K eMBB users, and set $\mathcal{N}$ of N URLLC users [1,20]. There are a set $\mathcal{B}$ of B RBs. The timeline is divided into T timeslots each containing M mini-slots. The channel peak rates for each user are monitored via CSI. For the purposed of this study we assume perfect CSI information is available in order to isolate and evaluate the performance of the proposed resource allocation algorithm under ideal conditions. eMBB requests are scheduled on millisecond-long Transmission Time Intervals (TTIs) while URLLC traffic is sent on shorter TTIs (0.125 ms) by puncturing existing eMBB transmissions. Each mini-slot can contain up to one packet of length ζ for each URLLC user. URRLC packets arrival are modeled as an i.i.d. random process and are not queued for later transmission [8]. We assume the case where eMBB are assigned individual RBs without slot aggregation. However, the 5G standard proposes more flexible aggregation and splitting schemes [8]. A summary of notations is presented in Table 3. Fig. 1 shows how URLLC users punctures the transmission of eMBB users

Table 3
Summary of notations.

Notation	Definition
$\mathcal{K}, \mathcal{N}, \mathcal{B}$	Set of eMBB users, URLLC users, and RBs, respectively.
K, N, B	$\lvert\mathcal{K}\rvert, \lvert\mathcal{N}\rvert, \lvert\mathcal{B}\rvert$
x	RB allocation matrix
z	RB puncturing matrix
x^t	Slice of x for at time t.
$\hat{z}$ *or* z^t	The slice of z for the current TTI.
$x_{kb}(t)$	RBs allocation variable for $k \in \mathcal{K}$ and $b \in \mathcal{B}$.
$p_{kb}(t)$	Power allocation variable for $k \in \mathcal{K}$ and $b \in \mathcal{B}$.
$z_b(t)$	Puncturing variable for $b \in \mathcal{B}$ at time t.
$y^{\mu}_{(t,s)}$	RBs allocation variable for $u \in \mathcal{N}$ at time sth mini-slot of time t.
$r^e_k(t)$	Data rate of eMBB user k at time slot t.
$r^e_{kb}(t)$	Data rate of eMBB user k on RB b at time slot t.
$r^{\mu}_{nb}(t)$	Data rate of URLLC user u on RB b at time slot t.
$r^{\mu}_{n}(t)$	Data rate of URLLC user n at time slot t.
$h^e_{kb}(t)$	eMBB channel gain for $k \in \mathcal{K}$ and $b \in \mathcal{B}$.
$r_b(t)$	The data rate of RB b at time t.
$\delta_b(t)$	Sum of data rate of URLLC users on RB b in one sTTI of t.
$r^{w}_b(t)$	The weighted data rate of RB b at time t.
$h^{\mu}_{nb}(t)$	URLLC channel gain for $n \in \mathcal{N}$ and $b \in \mathcal{B}$.
$p^{\mu}_{nb}(t)$	URLLC transmission power for $n \in \mathcal{N}$ and $b \in \mathcal{B}$.
$L(t)$	Total number of URLLC packets at a time slot t.
$c^{\mu}_{nb}(t)$	Length of a CB in symbols, for $n \in \mathcal{N}$, and $b \in \mathcal{B}$.
D^{μ}_{nb}	Channel dispersion at time slot t.
P_{max}	maximum transmission power.
$w_k(t)$	The weight assigned to eMBB user k at time t.
$u_b(t)$	The eMBB user that is assigned to RB b at time t.
M	Number of mini-slots in a time slot.
Δ	Duration of a TTI in seconds.
Θ_{max}	Maximum allowed outage probability for URLLC traffic.
ζ	URLLC packet size.
f_b	Bandwidth of RB b.
σ^2	Noise power.
$\mathcal{G}$	A graph in GNN
$\mathcal{V}, \mathcal{E}$	Set of graph nodes and edges respectively
u, v	Graph nodes.
V	number of graph nodes.
X, H	Feature and representation matrix of graph nodes respectively.
C, F	Feature and representation dimensions of node respectively.
$\mathbf{h}^{(l)}_k$	Hidden state of node k at GNN layer l.
$\mathbf{m}^{(l)}_{vu}$	The message sent from node v to u at GNN layer l.
$\mathbf{a}^{(l)}_v$	Aggregated hidden state of neighbors of v at GNN layer l
$\mathcal{N}_v$	Set of neighbors of node graph v.
$\mathcal{W}^{(l)}$	Learnable weights of MLP at layer l.
$\epsilon^{(l)}$	Learnable weight of hidden state update at layer l.
μ_e, σ_e	Mean and standard deviation of eMBB data rate.
μ_u, σ_u	Mean and standard deviation of URLLC data rate.
τ^e_k	eMBB per user gain.
λ_u	Rate of Poisson distribution of URLLC packet inter-arrival times.

3.1. eMBB data rate

The gNB schedules URLLC request on the immediate mini-slot by puncturing some RBs that are currently being used for eMBB transmission, as a result, the affected eMBB users will experience a rate loss relative to the number of punctured mini-slots [5]. The rate loss of eMBB due to puncturing is assumed to be linear [8]. Suppose $z_b(t)$ is the number of mini-slots punctured on RB b, then based on Shannon's capacity theory, the achievable data rate of eMBB user k on RB b at time t is approximated by Eq. (1) [1].

$$r^e_{kb}(t) = f_b \left(1 - \frac{z_b(t)}{M}\right) log_2 \left(1 + \frac{p_{kb}(t) h_{kb}(t)}{\sigma^2}\right) \tag{1}$$

where f_b is the bandwidth of the RB b and there are M mini-slots in each TTI. $p_{kb}(t)$ and $h_{kb}(t)$ are the transmission power and channel gain of eMBB user k on RB b respectively and σ^2 is the noise power. The total data rate of an eMBB user over all allocated RBs is given by (2).

$$r^e_k(t) = \sum_{b \in \mathcal{B}} x_{kb}(t) r^e_{kb}(t), \tag{2}$$

where $x_{kb}(t)$ is a binary variable for RB to eMBB user allocation. which is 1 if RB b is allocate to eMBB user k at time t and zero otherwise.

3.2. URLLC data rate

To satisfy the delay requirements of URLLC the block length should be short [30] therefore rate does not follow the Shannon's capacity

theory. In this situation, the finite block length coding should be applied. The rate of URLLC user n on RB b is given by (3) [1].

$$r_n^u(t) = \sum_{k\in\mathcal{K}}\sum_{b\in\mathcal{B}} \frac{f_b x_{kb}(t) z_b(t)}{M\times N} \log_2\left(1+\frac{p_{nb}^u(t) h_{nb}^u(t)}{\sigma^2}\right) - \sqrt{\frac{D_{nb}^u(t)}{c_{nb}^u(t)}} Q^{-1}(\vartheta) \tag{3}$$

where $Q^{-1}(\cdot)$ is the inverse Gaussian Q-function, ϑ is the transmission error probability, $c_{nb}^u(t)$ is the number of symbols for URLLC user n on RB b at time t, and $D_{nb}^u(t)$ is the channel dispersion for URLLC user n on RB b at time t given by (4).

$$D_{nb}^u(t) = 1 - \frac{1}{\left(1+\frac{p_n^u(t) h_n^u(t)}{\sigma^2}\right)^2} \tag{4}$$

4. Problem definition

At the start of each TTI, resource blocks are assigned to eMBB users so that long-term utilities such as total throughput and fairness are maximized. Resource allocation algorithms should take into account channel characteristics that can change over time and opportunistically identify the optimal resource block for each user. The algorithm should also consider the effect that URLLC puncturing can have on the perceived rate that eMBB users experience, resulting in a different rate than was originally predicted. Here we adopt the widely used mix-max fairness doctrine, which can simultaneously maximize the data rate and minimize its variance among users, contributes to an stable service quality, enhances spectral efficiency and overall user experience [20]. The objective function for eMBB allocation is formulated by (5) which should be maximized.

$$\mathcal{F}(x,z) = \min_{k\in\mathcal{K}} \sum_{t=1}^{T} r_k^e(t) \tag{5}$$

On the other hand, as URLLC traffic have the highest priority, all of URLLC requests must be served. Assume $L(t)$ is a random variable denoting number of URLLC request arriving at time t and Θ^{max} is the maximum allowed outage probability. A URLLC user is served if it is allocated enough RBs to transmit its ζ-bit message over short time span of a mini-slot. To satisfy the URLLC reliability requirement the condition given by (6) should hold [1].

$$\Pr\left[\sum_{n\in\mathcal{N}} r_n^u(t) \le \zeta L(t)\right] \le \Theta^{max} \tag{6}$$

Therefore the joint eMBB and URLLC resource allocation can be formulated as the following optimization problem.

$$\underset{x^t,z^t}{\text{maximize}} \ \min_{k\in\mathcal{K}} \sum_{t'=1}^{t} r_k^e(t) \tag{7a}$$

$$s.t. \ \Pr\left[\sum_{n\in\mathcal{N}} r_n^u(t) \le \zeta L(t)\right] \le \Theta^{max} \tag{7b}$$

$$\sum_{k\in\mathcal{K}} x_{kb}(t) \le 1, \ \forall\ b\in\mathcal{B} \tag{7c}$$

$$x_{kb}(t)\in\{0,1\}, \ \forall\ k\in\mathcal{K}, b\in\mathcal{B}, \tag{7d}$$

$$z_b(t)\in\{0,1,\dots,M\}, \ \forall\ b\in\mathcal{B}. \tag{7e}$$

This optimization problem attempts to find the optimum allocation of resource blocks to eMBB users (x^t) and the optimum number of punctured slots in all blocks (z^t). The objective is to maximize the minimum data rate of eMBB users, ensuring both total data rate and fairness. Probability constraints (7)b ensure URLLC reliability requirements, constraint (7)c states that each resource block is allocated to at most one eMBB user, constraint (7)d declares that $x_{kb}(t)$ is a binary variable and constraint (7)d ensures that the number of punctured slots per RB is an integer less than M. However, the optimization problem in Eq. (7) is an $\mathcal{NP}$-hard problem [1], due to it being a mixed-integer nonlinear programming (MINLP) problem [1]. To overcome this, we propose two phased resource allocation algorithms based on heuristics and GNNs, covered in the following sections.

5. Proposed solution

In practice, the eMBB and URLLC resource allocations are solved separately and repeatedly at different time scales. Therefore, the problem is divided into two separate sub-problems [6]. The first sub-problem is the eMBB resource allocation. At the time of eMBB allocation, no URLLC requests have been made yet, so constraint (7)b in problem (7) is removed. The eMBB resource allocation can be defined as (8).

$$\underset{x^t}{\text{maximize}} \ \min_{k\in\mathcal{K}} \left[\sum_{t'=1}^{t} r_k^e(t)\right] \tag{8a}$$

$$s.t. \ \sum_{k\in\mathcal{K}} x_{kb}(t) \le 1, \forall b\in\mathcal{B} \tag{8b}$$

$$x_{kb}(t)\in\{0,1\}, \forall\ k\in\mathcal{K}, b\in\mathcal{B}, \tag{8c}$$

where the URLLC constrain is removed and the problem only aims at finding the optimum x^t matrix. Therefore the second sub-problem which is the URLLC allocation problem considering the same objective function and a fixed x matrix is defined in (9).

$$\underset{z^t}{\text{maximize}} \ \min_{k\in\mathcal{K}} \left[\sum_{t'=1}^{t} r_k^e(t)\right] \tag{9a}$$

$$s.t. \ \Pr\left[\sum_{n\in\mathcal{N}} r_n^u(t) \le \zeta L(t)\right] \le \Theta^{max} \tag{9b}$$

$$z_b(t)\in\{0,1,\dots,M\}, \forall\ k\in\mathcal{K}. \tag{9c}$$

5.1. Heuristic solution for eMBB resource allocation

Sub-problem (8) is computationally expensive due to its $\mathcal{NP}$-hard nature [5,20], and finding the global optimum is difficult. On the other hand to solve the resource allocation problem in a single time slot, we need to formulate an approach that focuses on finding an optimum x^t slice from the x^t matrix that corresponds to the current time slot. We can express this problem using the weighted sum notation, that offers multiple trade-offs between sum-rate performance and user fairness [31]. This approach generalizes the min–max fairness, and is equal to it with the appropriate choice of weights [32]. The proposed heuristic is to solve the problem (10) instead.

$$\underset{x^t}{\text{maximize}} \ \sum_{k\in\mathcal{K}} w_k(t) r_k^e(t) \tag{10a}$$

$$s.t. \ \sum_{k\in\mathcal{K}} x_{kb}(t) \le 1, \forall\ b\in\mathcal{B} \tag{10b}$$

$$x_{kb}(t)\in\{0,1\}, \forall\ k\in\mathcal{K}, b\in\mathcal{B}, \tag{10c}$$

where x^t is the slice of x matrix corresponding to the current time slot. and $w_k(t)$ is the weight assigned to user $k\in\mathcal{K}$ at time slot t defined by (11). This will give greater weight to starved users and hence is expected to maximize the minimum total data rate of eMBB users through T time slots.

$$w_k(t) = \frac{1}{\sum_{t'=1}^{t-1} r_k^e(t') + \epsilon^1} \tag{11}$$

where $0<\epsilon^1\ll 1$ is to avoid division by zero. We propose the algorithm shown as Algorithm 1 to solve the problem (10). This algorithm finds the global optimum solution to the problem in polynomial time. Its optimality and time complexity are proven through the following theorems.

Theorem 1. *Algorithm 1 outputs the global optimum for problem (12)*

Algorithm 1 Heuristic Algorithm for eMBB resource allocation

Input: $\mathcal{K}, \mathcal{B}, r^e_{kb}(t)\ \forall\ k \in \mathcal{K}, b \in \mathcal{B}, w_k(t)\ \forall\ k \in \mathcal{K}$
Output: x^t
Initialization :
1: $x_{kb}(t) := 0\ \forall\ k \in \mathcal{K}, b \in \mathcal{B}$
2: **for all** $b \in \mathcal{B}$ **do**
3: $k := \underset{k \in \mathcal{K}}{\text{argmax}}(w_k(t) r^e_{kb}(t))$
4: $x_{kb}(t) := 1$
5: update $w_k(t)$
6: **end for**
7: **return** x^t

Proof. by substituting $r^e_k(t)$ from (2) and re-arranging the summations, problem (10)a will turn into:

$$\underset{x^t}{\text{maximize}} \quad \sum_{b \in \mathcal{B}} \sum_{k \in \mathcal{K}} x_{kb}(t) w_k(t) r^e_{kb}(t) \tag{12}$$

Note that according to (10)b, $x_k b(t)$ is 1 for only one k for each b. we define the weighted datarate of an RB $r^w_b(t)$ when assigned to a specif UE as:

$$r^w_b(t) = \sum_{k \in \mathcal{K}} x_{kb}(t) w_k(t) r^e_{kb}(t) \tag{13}$$

which is simply the data rate $r^e_{kb}(t)$ of the user k that is assigned to RB b when solution x^t is applied, multiplied by the fixed weight of k. By substituting this into (12) it turns into:

$$\underset{x^t}{\text{maximize}} \quad f = \sum_{b \in \mathcal{B}} r^w_b(t) \tag{14}$$

f will be maximized if and only if r^w_b is maximized for each RB. Note that because there is no constraint on the number of RBs that are assigned to each eMBB UE, each RB can independently choose the UE for which its data rate is maximum. i.e:

$$u_b(t) = \underset{k \in \mathcal{K}}{\text{argmax}}(w_k(t) r^e_{kb}(t)) \tag{15}$$

where $u_b(t)$ is the eMBB user assigned to RB b. Eq. (15) reflect the fact that each RB will choose the eMBB user with the maximum weighted data rate on that specific RB. Algorithm 1 will use this at line 3 to assign each RB to the most appropriate eMBB user, therefore it produces the solution with maximum f. □

The choice of weights for each UE in (11), with the goal of maximizing fairness and total data rate is done according to the widely used min–max fairness paradigm. Here we show that by assigning $w_k(t) = 1$ to all users, Algorithm 1 will output the solution whose total and average data rate among eMBB users are maximum, therefore a feasible upper-bound for data rate. This is useful for evaluating our choice of weight against the upper-bound to data rate, as a useful upper-bound for fairness is trivial (i.e. 1). The following theorem discusses that.

Theorem 2. *Algorithm 1 produces a feasible upper-bound for average and total eMBB data rate if* $w_k(t) = 1\ \forall\ \ k \in \mathcal{K}$

Proof. Theorem 1 proved that Algorithm 1 finds the optimal solution for (10) regardless of weights. Therefore it solves the problem also for $w_k(t) = 1\ \forall\ \ k \in \mathcal{K}$. if weights are 1, function f will become:

$$\underset{x^t}{\text{maximize}} \quad \sum_{k \in \mathcal{K}} r^e_k(t) \tag{16}$$

As $\sum_{k \in \mathcal{K}} r^e_k(t)$ is the definition of *total data rate*. Maximizing (16) is maximizing total data rate, and the average data rate is the total data rate divided by a constant. Therefore Algorithm 1 find a feasible solution with an upper-bound for total and average data rate if $w_k(t) = 1\ \forall\ \ k \in \mathcal{K}$. □

Theorem 3. *Time complexity of Algorithm 1 is* $\mathcal{O}(n)$ *where* $n = |\mathcal{B}| \times |\mathcal{K}|$

Proof. the first line initializes $x_{kb}(t)$ to 0 for all $|\mathcal{K}|$ users and $|\mathcal{B}|$ RBs, it is done in $\mathcal{O}(|\mathcal{B}| \times |\mathcal{K}|)$. The *for* loop at line 2 is run $|\mathcal{B}|$ times. In line 2 the argmax is found in $\mathcal{O}(|\mathcal{K}|)$ as finding the maximum among n numbers takes $\mathcal{O}(n)$ time. line 4 and 5 are setting a single value which are in $\mathcal{O}(1)$. Therefore what is inside the *for* loop take $\mathcal{O}(|\mathcal{K}|)$, this is multiplied by the number of loop iterations which is $|\mathcal{B}|$, so total time of the *for* loop is $\mathcal{O}(|\mathcal{B}| \times (|\mathcal{K}| + 2))$. Therefore the total time of algorithm is

$$\mathcal{O}(|\mathcal{B}| \times |\mathcal{K}|) + \mathcal{O}(|\mathcal{B}| \times (|\mathcal{K}| + 2)) = \mathcal{O}(|\mathcal{B}| \times |\mathcal{K}|) = \mathcal{O}(n) \quad \square \tag{17}$$

5.2. GNN for URLLC resource allocation

Sub problem (9) is essentially finding an optimum number of punctured mini-slots z for each RB, this can be viewed as a node representation learning problem to be solved by a GNN. To this end we construct a graph in which every node co-responds to a RB then train a GNN to compute the optimum z for each node given the state of the wireless network. Let us first formally define the node representation learning problem on graphs.

Problem Definition. Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ denote a graph consisting of a set of nodes $\mathcal{V}$ and a set of edges $\mathcal{E} \subset \{(v,u) | v,u \in \mathcal{V}\}$. $A \in \mathbb{R}^{V \times V}$ is the graph adjacency matrix, where V is total number of nodes. Also the graph has a node feature matrix $X \subset \mathbb{R}^{V \times C}$, where C is the number of features for each node. Node representation learning is the problem of learning effective node representations $H \subset \mathbb{R}^{V \times F}$ (where F is the dimension of node representation), by combining the node features and the graph structure attributes to be used by downstream tasks such as node classification and regression.

In the MPNN framework, multiple layers of message passing are stacked on top of each other. The basic idea of MPNN is that relevant information about the nodes, edges, and the entire graph can be encoded in fixed-size vectors [16]. Denote $\mathbf{h}^{(l)}_k$ as the hidden state of the kth node in the lth layer. Starting form the initial node representation $H^0 = X$, each layer consists of three stages:

- **Message**: Each node sends its hidden state to its neighbors, given by (18)a.
- **Aggregate**: Aggregation of information from neighbors of each node, given by (18)b.
- **Combine**: Updating representation of nodes by combining the aggregated information from neighbors with the current node representation, given by (18)c.

The above step can be formulated as (18) and depicted in Fig. 2.

$$MSG: \ \mathbf{m}^{(l)}_{vu} \leftarrow \text{MSG}(\mathbf{h}^{l-1}_v, \mathbf{h}^{l-1}_u), \forall (v,u) \in \mathcal{E} \tag{18a}$$

$$AGG \ \mathbf{a}^{(l)}_v \leftarrow \text{AGG}(\{\mathbf{m}^{(l)}_{vu} | u \in \mathcal{N}_v\}, \forall v \in \mathcal{V}), \tag{18b}$$

$$UPT: \ \mathbf{h}^{(l)}_v \leftarrow UPT(\mathbf{h}^{(l-1)}_v, \mathbf{a}^{(l)}_v), \forall v \in \mathcal{V}. \tag{18c}$$

where $\mathcal{N}_v$ is the set of neighbors of node v. The operations MSG, AGG, and UPT which stand for *message*, *aggregate*, and *update* respectively, are usually implemented via neural networks. Once we have the node representations from the last layer, a *readout* function can be applied to extract the final labels for nodes which is formulated in (19). The whole network can be trained to minimize a suitable loss function using the back propagation algorithm.

$$\hat{z}' = \text{READOUT}(\{\mathbf{h}^{(-1)}_v | \in \mathcal{V}\}). \tag{19}$$

where $\hat{z}'$ is a vector containing real numbers on range $[0,1]$, and $\mathbf{h}^{(-1)}_v$ is the hidden state of node v at the last layer. To obtain the final $\hat{z}$ this vector is multiplied by M and *floor* operator converts the real number to an integer which is interpreted as the number of mini-slots in which each RB is punctured given by (20).

$$z_b(t) = \lfloor \hat{z'}_b \times M \rfloor \ \forall\ b \in \mathcal{B} \tag{20}$$

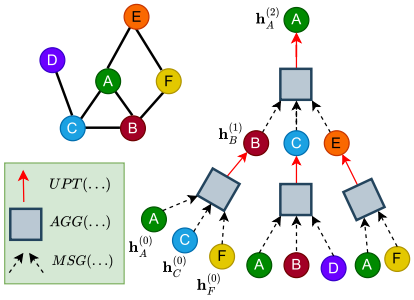


Fig. 2. The computing flow of MP-GNN to obtain a node representation.

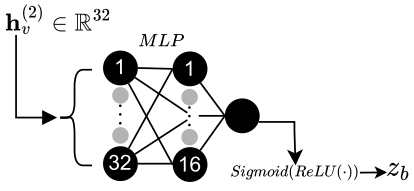


Fig. 3. The structure of the readout neural network.

To model (9) as a node representation learning problem, We construct a complete graph where each node represents a RB. Formally, the set of nodes is denoted as $\mathscr{V} = \mathcal{B}$, and the set of edges is $\mathscr{E} = (v,u); \forall; v,u \in \mathscr{V}$. This complete graph structure ensures that every pair of nodes (RBs) are connected, allowing the model to capture interactions between all RBs.

Each node in the graph has a feature vector that encapsulates relevant information about the RB it represents. The node feature matrix $X \in \mathbb{R}^{|\mathscr{V}|\times 3}$ is defined such that each node $v \in \mathscr{V}$ has a feature vector $X^v = \langle r_b(t), \delta_b(t), \zeta L(t)\rangle$. Where $r_b(t)$ is the data rate assigned to eMBB on RB b at time slot t, calculated as per Eq. (21). $\delta_b(t)$ represents the total data rate of URLLC users on RB b, computed as per Eq. (22). $\zeta L(t)$ Is the total size of packets for all URLLC user at time t which capture the total data rate required to serve all URLLC users. Node features vector, is given by (23). In this representation, the GNN learns to aggregate information from neighboring nodes (RBs) and updates the hidden states of nodes, which helps in finding optimal resource allocation configurations for incoming URLLC users. The complete graph structure enables the model to evaluate the impact of assigning different puncturing weights $z_b(t)$ to any RB, considering both the data rate loss experienced by eMBB users and the data rate achieved by URLLC users.

$$r_b(t) = \sum_{k\in\mathcal{K}} x_{kb}(t) r^e_{kb}(t) \tag{21}$$

$$\delta_b(t) = \sum_{u\in\mathcal{N}} r^u_{nb}(t) \tag{22}$$

$$\mathbf{h}^{(0)}_v = \langle r_b(t), \delta_b(t), \zeta L(t)\rangle \;\; \forall\; v \in \mathscr{V} \tag{23}$$

In the proposed GNN architecture we use MLPs for the message and update operators. According to universal approximation theorem [33] MLPs enjoy the property of being universal optimizer, thus an excellent tool to model MSG and UPT operators [13]. Furthermore, it is shown in [34] that we can combine the two MLPs used in MSG and UPT into one MLP without decreasing expressive power. Therefore the simplest update mechanism is obtained as given by (24) [13].

$$\mathbf{h}^{(l)}_{\mathbf{v}} = MLP^{(l-1)}\left[(1+\epsilon^{(l)})\mathbf{h}^{(l-1)}_v + \sum_{u\in\mathcal{N}_v} \mathbf{h}^{(l-1)}_u | \mathcal{W}^{(l-1)}\right], \;\; \forall\; v \in \mathscr{V} \tag{24}$$

where $\mathcal{W}^{(l-1)}$ and $\epsilon^{(k)}$ are the learnable weights. In a complete graph, this update mechanism removes the need for each node to iterate over all other nodes to update its hidden state, lowering time complexity of the algorithm as we discuss later. In our network, the $READOUT$ operation is also a MLP. To construct the GNN, we used two layers of message passing followed by a readout network to obtain z_b for each RB. The MLP used in the first layer has 6 (3 for each node along an edge) input and 16 output neurons and the MLP used in the second layer has 32 (16 for each node along an edge) input and 32 output neurons. Both MLPs use ReLU as the activation function. And the architecture of the readout MLP is depicted in Fig. 3.

Algorithm 2 GNN-based puncturing RB puncturing

Input: $r_b(t), \delta_b(t), L(t) \;\forall\; b \in \mathcal{B},\; \zeta, \mathscr{V}$
Output: z^t
1: $\mathbf{h}^{(0)}_{v=b} = \langle r_b(t), \delta_b(t), \zeta L(t)\rangle \;\; \forall\; v \in \mathscr{V}$
2: **for** l in {1,2} **do**
3: $\quad \mathbf{h}_{all} = \sum_{v\in\mathscr{V}} \mathbf{h}^{(l-1)}_u$
4: $\quad$ **for all** $v \in \mathscr{V}$ **do**
5: $\quad\quad h^{sum}_{\mathcal{N}_n} = \mathbf{h}_{all} - \mathbf{h}^{(l-1)}_v$
The graph is compete thus:
$\sum_{u\in\mathcal{N}_v} \mathbf{h}^{(l-1)}_u = \mathbf{h}_{all} - \mathbf{h}^{(l-1)}_v$
according to (24):
6: $\quad\quad mlp_input := (1+\epsilon^{(l)})\mathbf{h}^{(l-1)}_v + h^{sum}_{\mathcal{N}_n}$
7: $\quad\quad \mathbf{h}^{(l-1)}_v = MLP^{(l-1)}(mlp_input|\mathcal{W}^{(l-1)})$
8: $\quad$ **end for**
9: **end for**
10: $\hat{z'} = \text{READOUT}(\{\mathbf{h}^{(L=2)}_v | \in \mathscr{V}\})$.
11: $z_b(t) = \lfloor \hat{z'}_b \times M \rfloor \;\forall\; b \in \mathcal{B}$
12: **return** z^t

Theorem 4. *Time complexity of the GNN-based method shown as Algorithm 2 is $\mathcal{O}(n)$ where n is the number of RBs.*

Proof. We consider the time complexity of the algorithm by going through the pseudo code. At line 1, $\mathbf{h}^{(0)}_v$ is constructed for each node of the graph, and as the nodes are RBs, this line is completed in $\mathcal{O}(n)$. At line 2, a *for* loop that repeats 2 times starts. Line 3 computes h_{all} for each node that is also done in $\mathcal{O}(n)$. What is inside the loop at line 4 repeats for n time and therefore their total time complexity is multiplied by n. At line 5 two fixed sized vectors are subtracted in $\mathcal{O}(1)$. Line 6 is also an algebraic operation on fixed sized vectors with $\mathcal{O}(1)$ time complexity. The MLP is applied at line 7, because number of MLP layers and input/output neurons in each layer are fixed and do not grow with size of the problem, i.e., n, it can be regarded as a constant. As the time complexity of constant-sized computation is $\mathcal{O}(1)$ the complexity of the MLPs can be considered $\mathcal{O}(1)$, same applies to line 10, therefore total time complexity is (25). The line numbers are reflected below each term or factor.

$$\begin{aligned}&\underset{line1}{\mathcal{O}(n)} + \underset{line2}{2} \times (\underset{line3}{\mathcal{O}(n)} + \underset{line4}{n} \times (\underset{line5}{\mathcal{O}(1)} + \underset{line6}{\mathcal{O}(1)} + \underset{line7}{\mathcal{O}(1)})) + \underset{line10}{\mathcal{O}(1)} \\ &\quad = \mathcal{O}(n) + 2 \times \mathcal{O}(4n) + \mathcal{O}(1) \\ &\quad\quad = \mathcal{O}(9n+1) = \mathcal{O}(n). \;\square\end{aligned} \tag{25}$$

In order to train the network and because we do not have any labeled data, we designed a suitable application-specific loss function. As the optimization goal of problem (9) is to maximize the total data rate of eMBB UEs, we can define loss function as $-\sum_{k\in\mathcal{K}} r^e_k(t)$, note that the negative sign is because the training algorithm will minimize the loss while we aim to maximize the total data rate. One concern however is missed in this formulation that is the URLLC reliability. recalling from (6), a threshold for total URLLC data rate is $\zeta L(t)$, therefore we incorporate this using a soft-threshold function such as $sigmoid$ into the loss function as normal threshold function is not differentiable. We have used the loss function given by (26) to train the neural network.

$$\text{loss}(z^t) = -\sum_{k\in\mathcal{K}} r^e_k - \text{sigmoid}(\sum_{n\in\mathcal{N}} r^u_n - \zeta L(t)) \times W_{penalty} \tag{26}$$

where $W_{penalty}$ is used to add a large penalty in case the total data rate of URLLC users falls below the threshold, to ensure URLLC reliability. The model is programmed using PyTorch and the Deep Graph Library (DGL) [35], and the Adam optimizer is used. The goal of the model is to predict a number of each node of the graph. The number in question is the number of punctured mini-slots for the RB (z_b) normalized to range $[0, 1]$. It is converted to an integer in range $[0, M]$ based on (20). After the model assigned a z to each RB, upon arrival of URLLC traffic, the packet is scheduled on the best RBs that has remaining non-punctured mini-slots. Algorithm 3 clarifies in process in detail.

Algorithm 3 RB allocation to URLLC users at mini-slots of TTI t

Output: $y_{(t,*)}$
For each sTTI in the current time slot:
for $s \in \{1, \dots, M\}$ **do**
  $\mathcal{N}_s$ = arrived URLLC users at mini-slot s
  for $u \in \mathcal{N}_s$ **do**
    $\mathcal{B}_p = \{b \mid b \in \mathcal{B} \wedge z_b(t) > 0\}$
    if $\mathcal{B}_p = \emptyset$ **then**
      $\mathcal{B}_p = \mathcal{B}$
    end if
    While not enough data rate is allocated to u:
    $r = 0$ data rate allocated to u so far
    while $(r \times \Delta) < \zeta$ **do**
      $b = \underset{b' \in \mathcal{B}_p}{\operatorname{argmax}}(h^u_{nb'}(t))$
      if b is none **then**
        break
      end if
      $\mathcal{B}_p = \mathcal{B}_p - \{b\}$
      $z_b(t) = z_b(t) - 1$
      $r = r + r^u_{nb}(t)$
      $y^u_{(t,s)} = b$
      update $r^u_n(t)$
    end while
  end for
end for
return $y_{(t,*)}$

6. Performance evaluation

In this section we validate the effectiveness of our approach via comprehensive experimental analysis. All simulations are programmed in python using PyTorch and the Deep Graph Library [35] and ran on a desktop PC equipped with a quad-core 2.4 GHz Intel Core i5 processor. The main metrics we considered where data rate of eMBB users, fairness among them and the URLLC outage probability. For fairness we used the Jain's fairness as given by (27), where $\mathcal{R}$ is a set of data rate values. Also to achieve more reliable results, simulations have been repeated for 10 times.

$$\text{fairness}(\mathcal{R}) = \frac{\left(\sum_{r\in\mathcal{R}} r\right)^2}{|\mathcal{R}| \sum_{r\in\mathcal{R}} r^2} \tag{27}$$

6.1. Network and traffic modeling

we simulated downlink requests in a network consists of a gNB and a variable number of URLLC and eMBB users to assess different scenarios. The fully buffered traffic model, which is widely used in network simulation [1], was used for the eMBB users as in [1,5,6,20]. In this model, it is assumed that all users always have data in their input buffers. The gNB is assumed to know the data rate of all user on all RBs through means of CSI. The data rate of eMBB users are simulated as a time varying and normally distributed random variables in such a way that some users have overall better conditions throughout the time-span of the simulation. This is to ensure that our proposed method considers users in poor condition and does a fair job in resource allocation. The mean of eMBB users data rate distribution is μ_e with the standard deviation σ_e. This random value is multiplied by a normally distributed per-user gain τ^e_k to simulate advantageous and poor-conditioned users. In contrast to eMBB users, the URLLC users only have data in some mini-slots, the inter-arrival time of URLLC packets follow an i.i.d. Poisson distribution [1,36] with rate λ_u and a URLLC user may user multiple RBs in one mini-slot. The data rate of URLLC users on different RBs is modeled as a normally distributed random variable with parameters μ_u and σ_e. The size of URLLC data packets is 32 bytes [1]. Table 4 lists parameters of the simulation.

Table 4
Parameters of simulation.

Parameter	Value
$\lvert\mathcal{K}\rvert$	10–100
$\lvert\mathcal{N}\rvert$	10–100
$\lvert\mathcal{B}\rvert$	50
M	8
Δ	10^{-3}s (1 ms)
ζ	32 bytes
μ_e, σ_e	360 kbps, 180 kbps
μ_u, σ_u	150 kbps, 75 kbps
τ^e_k	$\sim \mathcal{N}(1, 0.5^2)$
λ_u	2 ms [36]
Learning rate	10^{-5}
$W_{penalty}$	10
$\lvert\mathcal{V}\rvert$ for training	50
$\lvert\mathcal{V}\rvert$ for testing	100

6.2. Benchmark comparisons

For performance comparison of our eMBB resource allocation scheme, we used two methods, One is a feasible upper-bound for data rate as proved by Theorem 2, and the other is the random method which provides close to theoretical upper-bound for fairness. For these reasons we did not compare our work to more works. The compared eMBB resource allocation schemes are:

- **Sum-Rate**: This is an upper-bound for the average and total eMBB data rate, thus a very suitable baseline to compare against. This approach used $\omega_k = 1\ \forall k \in \mathcal{K}$ for which Theorem 2 proved being upper-bound.
- **Random**: The random approach assigns eMBB users to RBs randomly in order to achieve high fairness, as is evident in the simulations it is very close to the theoretical upper-bound of fairness=1 thus a suitable method to compare the fairness of our scheme against.

For the URLLC puncturing evaluation we implemented four other preemption schemes. To make the comparisons more reliable, we used our proposed eMBB scheduler for all the compared URLLC puncturing schemes except for Prathyusha et al. which propose their own eMBB scheduler. These are listed below.

- **Random**: The arrived URLLC user randomly chooses an RB, this is expected to achieve a good fairness for eMBB users and is used as a baseline for fairness.
- **Greedy**: The arrived URLLC user will take many RBs as it need starting from the best RBs having the highest data rate for that specific URLLC user
- **Min-Rate**: The arrived URLLC user will be assigned to the RB for which the corresponding eMBB user has the lowest data rate.

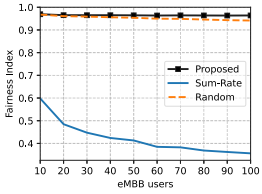


Fig. 4. The Jain's fairness index.

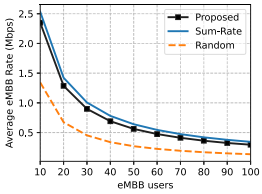


Fig. 5. Average eMBB data rate.

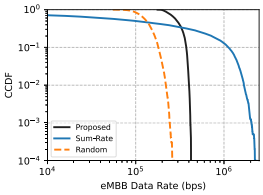


Fig. 6. CCDF of the average eMBB data rate.

- **Prathyusha et al.** [23]: A puncturing and superposition based approach via one-to-one matching game model with the goal of maximizing the minimum expected achieved rate (MEAR) while also satisfying the URLLC service requirements. It employs a heuristic low complexity algorithm for the eMBB resource allocation to improve throughput and MEAR and a contract-based approach based on matching game for URLLC scheduling with the goal of retaining fairness among eMBB users.

6.3. eMBB resource allocation performance

First we discuss the effectiveness of our proposed eMBB resource allocation approach without any URLLC users present. The results of the simulation are shown in Figs. 4–5. Fig. 4 shows the fairness results of the proposed eMBB resource allocation algorithm. Our method has outperformed the Sum-Rate method significantly while also being better or equally as good as the random approach in terms of fairness. The reason behind this, is that our protocol gives higher weights to the starved users, in a way that a user that has gotten low data rate so far, will have a higher chance of being assigned multiple good RBs in the following TTIs. The Random approach is also relatively fair dues to pure randomness.

In terms of average data rate among eMBB users, our algorithm preforms very close to the Sum-Rate method. Note that as the Sum-Rate method is a proven upper-bound, no other scheme may exceed that. While random approach have lower data rate and better fairness, our algorithm is very successful in consolidating the best of both worlds, i.e. having good fairness and close to upper-bound average data rate. Any other protocol can only improve upon our work in terms of average data rate up to the upper-bound which plotted as the Sum-Rate approach and is very close to our results.

Fig. 6 plots the complementary cumulative distribution function (CCDF) of eMBB data rate for a sample TTI in a network with 100 eMBB users. It shows the fairness of our method form another perspective. The crosspoint of the random and proposed methods indicates the point at which the proposed method outperforms the random approach. On the left side of this intersection, the random approach is more effective, meaning that a greater number of users are provided with data rates as indicated on the x-axis. However, on the right side of the crosspoint, the percentage of users with higher data rates in the random approach decreases rapidly, while the proposed approach maintains a higher percentage of users above the data rates specified on the x-axis. This same pattern can also be observed at the intersection of the sum-rate and proposed methods.

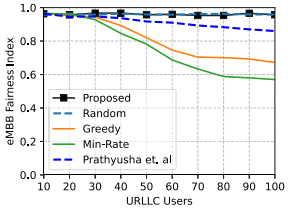


Fig. 7. The Jain's fairness index.

Table 5
Optimality gap of the proposed eMBB resource allocation algorithm.

Users	Data rate			Fairness index	
	Achieved	Optimal	Gap	Achieved	Gap
10	2.19	2.25	2.66%	0.99	1%
20	1.20	1.36	11.7%	0.96	4%
30	0.92	1.03	10.6%	0.97	3%
40	0.65	0.76	14.4%	0.96	4%
50	0.55	0.63	12.6%	0.96	4%
60	0.48	0.55	12.7%	0.96	4%
70	0.41	0.48	14.5%	0.96	4%
80	0.37	0.41	9.75%	0.97	3%
90	0.33	0.38	13.15%	0.97	3%
100	0.29	0.34	14.7%	0.96	4%

Table 5 contains the detailed optimality gap for in each of the simulation for the eMBB resource allocation algorithm. The optimal value for the data rate is that of the sum-rate approach as shown by Theorem 2. For the fairness an upper-bound is 1 and is the same for any number of users therefore does not have a dedicated column in Table 5.

6.4. eMBB and URLLC co-existing performance

In the next simulations, we evaluated the performance of our GNN-based URLLC resource allocation algorithm. A network of 100 eMBB users with varying number of URLLC users were simulated in order to test the effect of URLLC users on fairness and data rate of eMBB users. Fig. 8 plots the average data rate of eMBB users as the number of URLLC users grow. The greedy and Min-Rate method have the worse performance as the greedy method work completely in favor of URLLC user and the Min-Rate method sacrifices the already starved users to further lower the average data rate. The random method is a good baseline to infer the expected performance of a more intelligent method. The method proposed in [23] outperformed our method when the number of URLLC users were low but failed to keep showing competitive results as the URLLC users grow in number while the scalability of our GNN-based method enables it to perform well with higher number of URLLC users. Our GNN-based method surpasses all other scheme significantly, This shows the power of MLP augmented GNNs from two perspectives. Firstly the value of the average eMBB data rate is high, and secondly as the number of URLLC users grow, the GNN does not lose its efficiency and generalize very well. The gentle decline in average eMBB data rate is the inevitable result of increased URLLC traffic. As an example, for 10 URLLC users the average eMBB data rate is just below 0.7 Mbps, and for 10x URLLC users (100 users) it is only reduced to 0.6 Mbps.

Table 6
Optimality gap of the proposed URLLC resource allocation algorithm.

Metric	Achieved	Optimal	Gap
eMBB data rate	173 kbps	192 kbps	9.89%
eMBB fairness	0.81	0.87	6.8%

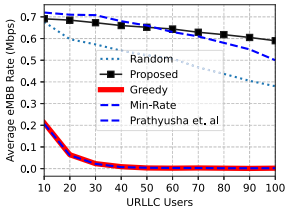


Fig. 8. Average eMBB data rate.

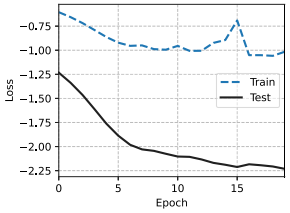


Fig. 9. Convergence of loss function for train and test data.

In order to assess the optimality gap of our proposed URLLC resource allocation method, we simulated a small topology for which computing an exact solution is possible. This network contains 5 RBs, 5 eMBB users and it is simulated for a single time slot with 15 URLLC users each having one request. Other parameters are the same as listed in Table 4. The size of solution space for this problem is 5^{15} or approximately 30 billion as we have to choose one out of 5 RB for each of 15 URLLC request. The optimum solution is then found by exhaustive search approach and compared to the outcome of our proposed GNN-based algorithm. These numeric results are listed in Table 6.

Also looking at the fairness plot in Fig. 7 it is evident that our method even outperforms the random approach and performs very close to perfect fairness, its 97% for the 100 URLLC users case. It also shows the power of our GNN-based method in multi-objective optimization problems. The most important metric for ULLC is the outage probability as the standard mandates a 99.999% success rate, meaning the maximum of 0.001% is allowed for the outage probability. Outage may occur when there is no remaining RBs to puncture due to inefficient RB to URLLC allocation as a single URLLC user might use multiple RBs, or the remaining RBs do not provide enough data rate for the URLLC packet to be transmitted in the very short duration of a mini-slot, which is also the result sub-optimal RB to URLLC allocation. Table 7 lists the results of outage probability for the compared methods for two scenarios having 100 and 150 URLLC users respectively. The random and min-rate approaches could not meet the requirements while greedy approach succeeds with for the price of substantially decreased average data rate of eMBB users. On the other hand, the proposed GNN-based puncturing algorithm is performing even better than what the standard mandates. Here again, the results of the random approach is a good indication that the URLLC traffic were high enough to cause outages above that of the specified by the standard. Ref. [23] also managed to show performance similar to the proposed method but did not match the performance of the proposed method when number of URLLC users increased to 150.

Table 7
Outage probability of different methods.

Method	Outage probability	
	100 URLLC users	150 URLLC users
Proposed	0.0006%	0.014%
Greedy	0.001%	0.033%
Random	0.0015%	0.038%
Min-Rate	0.0023%	0.05%
Prathyusha et al. [23]	0.0006%	0.021%

6.5. GNN generalization and convergence

Fig. 9 illustrates the convergence of the loss function for both the training and testing datasets during training. It is important to note that the test data comprises networks with 100 RBs, while the training data consists of networks with 50 RBs. This difference results in varying total data rates between the two datasets. Given that our loss function includes a negative sign, the larger total data rate in the testing dataset translates to lower (more negative) loss values.

This plot also demonstrates the generalization capability of the GNN, showing that a network trained on a 50-node graph performs effectively on a 100-node graph. The convergence of the proposed loss function is evident, with the loss reaching a stable minimum within just 20 epochs.

However, a significant increase in the training loss at epoch 15 is observed, which can be attributed to the complexity of our custom loss function and the dynamic learning rate adjustments by the Adam optimizer. These adjustments, coupled with the intricate nature of the loss function, occasionally lead to temporary increases in the loss during training.

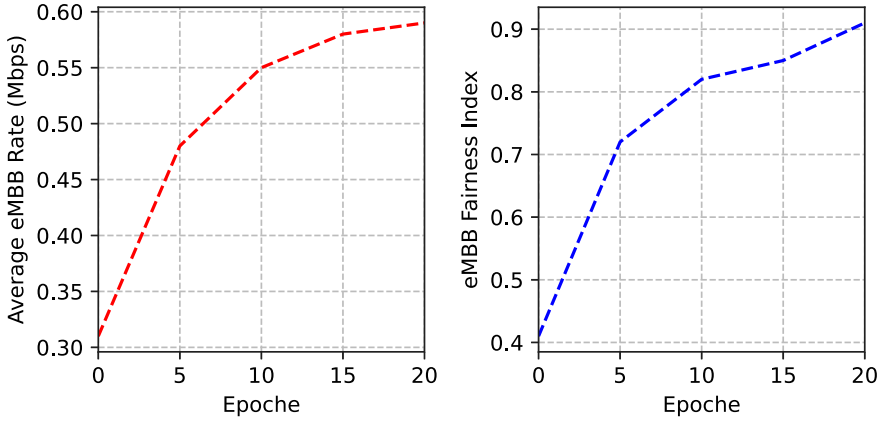


Fig. 10. The end-to-end convergence of eMBB data rate and fairness index in terms of training epochs.

The generalization ability of the GNN stems from its core mechanism: learning how to aggregate information from a node's neighbors and updating its hidden state. This learning process is uniform for all nodes in the graph, allowing the trained network to be applied regardless of the graph's size. Consequently, we conclude that the GNN is capable of managing networks with different topologies and traffic volumes, showcasing the expressiveness of the MLPs used within the GNN.

An additional advantage of our approach is the computational efficiency. Both the heuristic eMBB scheduler and the GNN-based puncturing algorithm operate in $\mathcal{O}(n)$ time, making them highly suitable for addressing these problems within the brief duration of mini-slots.

Fig. 10 plots the end-to-end convergence of the algorithm in terms of data rate and fairness for on a network with 100 eMBB and 100 URLLC users. This plot is generated by using the partly trained GNN to allocate resources to user while the GNN is being trained in different epochs until the final epoch in which the algorithm converges.

7. Conclusion

In this paper, we studied the coexisting problem of eMBB and URLLC services via the preemptive puncturing framework. We investigated how to allocate RBs to eMBB users at TTI time scale and proposed an algorithm with $\mathcal{O}(n)$ runtime for eMBB resource allocation. Then we turned our focus to the URLLC preemptive puncturing problem and presented a deep graph neural network model that we trained using an application specific loss function and showed that inference using this model is also of time complexity $\mathcal{O}(n)$. We also presented the simulation results of our proposed approach in terms of data rate, fairness, outage probability, loss convergence and GNN generalization. Comparing the result with a number of carefully chosen baselines reveals the usefulness of GNNs to tackle this challenge. In our future research we will investigate the effect of imperfect CSI and CSI estimation techniques on the architecture and performance of the proposed GNN.

CRediT authorship contribution statement

Seyyed Mohammad Mahdi Hosseini Daneshvar: Writing – original draft, Visualization, Software, Formal analysis. **Sayyed Majid Mazinani:** Writing – review & editing, Supervision, Methodology, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] Madyan Alsenwi, Nguyen H. Tran, Mehdi Bennis, Shashi Raj Pandey, Anupam Kumar Bairagi, Choong Seon Hong, Intelligent resource slicing for eMBB and URLLC coexistence in 5g and beyond: A deep reinforcement learning based approach, IEEE Trans. Wirel. Commun. 20 (7) (2021) 4585–4600.

[2] Antonio Matencio-Escolar, Qi Wang, Jose M. Alcaraz Calero, SliceNetVSwitch: Definition, design and implementation of 5G multi-tenant network slicing in software data paths, IEEE Trans. Netw. Serv. Manag. 17 (4) (2020) 2212–2225.

[3] Estefania Coronado, Shah Nawaz Khan, Roberto Riggio, 5G-EmPOWER: A software-defined networking platform for 5G radio access networks, IEEE Trans. Netw. Serv. Manag. 16 (2) (2019) 715–728.

[4] Erik Dahlman, Stefan Parkvall, Johan Skold, 5G NR: The Next Generation Wireless Access Technology, Academic Press, San Diego, CA, 2020.

[5] Anupam Kumar Bairagi, Md. Shirajum Munir, Madyan Alsenwi, Nguyen H. Tran, Choong Seon Hong, A matching based coexistence mechanism between eMBB and uRLLC in 5g wireless networks, in: Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing, ACM, 2019.

[6] Yan Huang, Shaoran Li, Chengzhang Li, Y. Thomas Hou, Wenjing Lou, A deep-reinforcement-learning-based approach to dynamic eMBB/URLLC multiplexing in 5G NR, IEEE Int. Things J. 7 (7) (2020) 6439–6456.

[7] Yan Huang, Y. Thomas Hou, Wenjing Lou, DELUXE: A DL-based link adaptation for URLLC/eMBB multiplexing in 5G NR, IEEE J. Selected Areas Commun. 40 (1) (2022) 143–162.

[8] Arjun Anand, Gustavo de Veciana, Sanjay Shakkottai, Joint scheduling of URLLC and eMBB traffic in 5G wireless networks, IEEE/ACM Trans. Netw. 28 (2) (2020) 477–490.

[9] Navid NaderiAlizadeh, Mark Eisen, Alejandro Ribeiro, Adaptive wireless power allocation with graph neural networks, in: ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), IEEE, 2022.

[10] Krzysztof Rusek, José Suárez-Varela, Albert Mestres, Pere Barlet-Ros, Albert Cabellos-Aparicio, Unveiling the potential of graph neural networks for network modeling and optimization in SDN, in: Proceedings of the 2019 ACM Symposium on SDN Research, ACM, 2019.

[11] Vinicius Lima, Mark Eisen, Konstatinos Gatsis, Alejandro Ribeiro, Resource allocation in large-scale wireless control systems with graph neural networks, IFAC-PapersOnLine 53 (2) (2020) 2634–2641.

[12] Yifei Shen, Jun Zhang, S.H. Song, Khaled B. Letaief, Graph neural networks for wireless communications: From theory to practice, IEEE Trans. Wirel. Commun. (2022) 1.

[13] Lingfei Wu, Peng Cui, Jian Pei, Liang Zhao (Eds.), Graph Neural Networks: Foundations, Frontiers, and Applications, Springer Nature Singapore, 2022.

[14] Xiao Xu, You Lu, Qiming Fu, Applying graph neural network in deep reinforcement learning to optimize wireless network routing, in: 2021 Ninth International Conference on Advanced Cloud and Big Data (CBD), IEEE, 2022.

[15] Tianrui Chen, Xinruo Zhang, Minglei You, Gan Zheng, Sangarapillai Lambotharan, A GNN-based supervised learning framework for resource allocation in wireless IoT networks, IEEE Int. Things J. 9 (3) (2022) 1712–1724.

[16] Krzysztof Rusek, Jose Suarez-Varela, Paul Almasan, Pere Barlet-Ros, Albert Cabellos-Aparicio, RouteNet: Leveraging graph neural networks for network modeling and optimization in SDN, IEEE J. Selected Areas Commun. 38 (10) (2020) 2260–2270.

[17] Shiwen He, Shaowen Xiong, Yeyu Ou, Jian Zhang, Jiaheng Wang, Yongming Huang, Yaoxue Zhang, An overview on the application of graph neural networks in wireless networks, IEEE Open J. Commun. Soc. 2 (2021) 2547–2565.

[18] Yifei Shen, Yuanming Shi, Jun Zhang, Khaled B. Letaief, Graph neural networks for scalable radio resource management: Architecture design and theoretical analysis, IEEE J. Selected Areas Commun. 39 (1) (2021) 101–115.

[19] Suzhi Bi, Liang Huang, Ying-Jun Angela Zhang, Joint optimization of service caching placement and computation offloading in mobile edge computing systems, IEEE Trans. Wirel. Commun. 19 (7) (2020) 4947–4963.

[20] Anupam Kumar Bairagi, Md. Shirajum Munir, Madyan Alsenwi, Nguyen H. Tran, Sultan S. Alshamrani, Mehedi Masud, Zhu Han, Choong Seon Hong, Coexistence mechanism between eMBB and uRLLC in 5G wireless networks, IEEE Trans. Commun. 69 (3) (2021) 1736–1749.

[21] Wenheng Zhang, Mahsa Derakhshani, Sangarapillai Lambotharan, Stochastic optimization of URLLC-eMBB joint scheduling with queuing mechanism, IEEE Wirel. Commun. Lett. 10 (4) (2021) 844–848.

[22] Mohammed Almekhlafi, Mohaned Chraiti, Mohamed Amine Arfaoui, Chadi Assi, Ali Ghrayeb, Amira Alloum, A downlink puncturing scheme for simultaneous transmission of URLLC and eMBB traffic by exploiting data similarity, IEEE Trans. Veh. Technol. 70 (12) (2021) 13087–13100.

[23] Yerra Prathyusha, Tsang-Ling Sheu, Coordinated resource allocations for eMBB and URLLC in 5G communication networks, IEEE Trans. Veh. Technol. 71 (8) (2022) 8717–8728.

[24] Aunas Manzoor, S. M. Ahsan Kazmi, Shashi Raj Pandey, Choong Seon Hong, Contract-based scheduling of URLLC packets in incumbent EMBB traffic, IEEE Access 8 (2020) 167516–167526.

[25] Mengyuan Lee, Guanding Yu, Huaiyu Dai, Geoffrey Ye Li, Graph neural networks meet wireless communications: Motivation, applications, and future directions, IEEE Wirel. Commun. 29 (5) (2022) 12–19.

[26] Qiushuo Hou, Mengyuan Lee, Guanding Yu, Yunlong Cai, Meta-gating framework for fast and continuous resource optimization in dynamic wireless environments, IEEE Trans. Commun. 71 (9) (2023) 5259–5273.

[27] Haozhe Wang, Yulei Wu, Geyong Min, Wang Miao, A graph neural network-based digital twin for network slicing management, IEEE Trans. Ind. Inform. 18 (2) (2022) 1367–1376.

[28] Ting Zhu, Xiaohui Chen, Li Chen, Weidong Wang, Guo Wei, GCLR: GNN-based cross layer optimization for multipath TCP by routing, IEEE Access 8 (2020) 17060–17070.

[29] Yuhong Liu, Changyang She, Wibowo Hardjawana, Branka Vucetic, Graph neural networks for timely updates of short packets in interference-limited networks, in: 2022 56th Asilomar Conference on Signals, Systems, and Computers, 2022, pp. 1050–1054, http://dx.doi.org/10.1109/IEEECONF56349.2022.10051893.

[30] Changyang She, Chengjian Sun, Zhouyou Gu, Yonghui Li, Chenyang Yang, H. Vincent Poor, Branka Vucetic, A tutorial on ultrareliable and low-latency communications in 6G: Integrating domain knowledge into deep learning, Proc. IEEE 109 (3) (2021) 204–246.

[31] Pradeep Chathuranga Weeraddana, Weighted sum-rate maximization in wireless networks: A review, Found. Trends Netw. 6 (1–2) (2011) 1–163.

[32] Atilla Eryilmaz, R. Srikant, Fair resource allocation in wireless networks using queue-length-based scheduling and congestion control, IEEE/ACM Trans. Netw. 15 (6) (2007) 1333–1344.

[33] Kurt Hornik, Maxwell Stinchcombe, Halbert White, Multilayer feedforward networks are universal approximators, Neural Netw. 2 (5) (1989) 359–366.

[34] Keyulu Xu, Weihua Hu, Jure Leskovec, Stefanie Jegelka, How powerful are graph neural networks? in: International Conference on Learning Representations, 2019, URL https://openreview.net/forum?id=ryGs6iA5Km.

[35] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, Tianjun Xiao, Tong He, George Karypis, Jinyang Li, Zheng Zhang, Deep graph library: A graph-centric, highly-performant package for graph neural networks, 2019, arXiv preprint arXiv:1909.01315.

[36] Changyang She, Rui Dong, Wibowo Hardjawana, Yonghui Li, Branka Vucetic, Optimizing resource allocation for 5G services with diverse quality-of-service requirements, in: 2019 IEEE Global Communications Conference (GLOBECOM), IEEE, 2019.